

CYBERSECURITY-LABS

INFORMATION SECURITY

LAB REPORT



Práctica: Linux Hardening

Informe de práctica

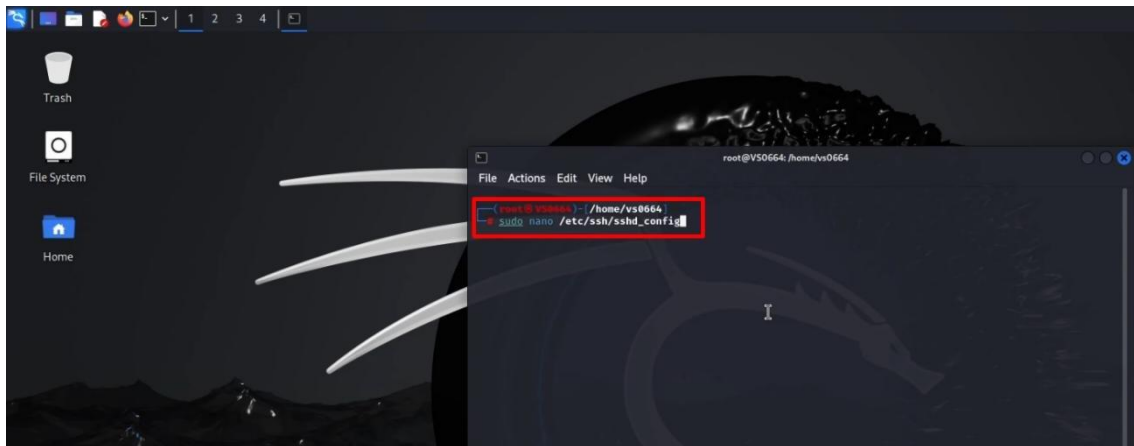
Víctor José Santos Guzmán

Information Security

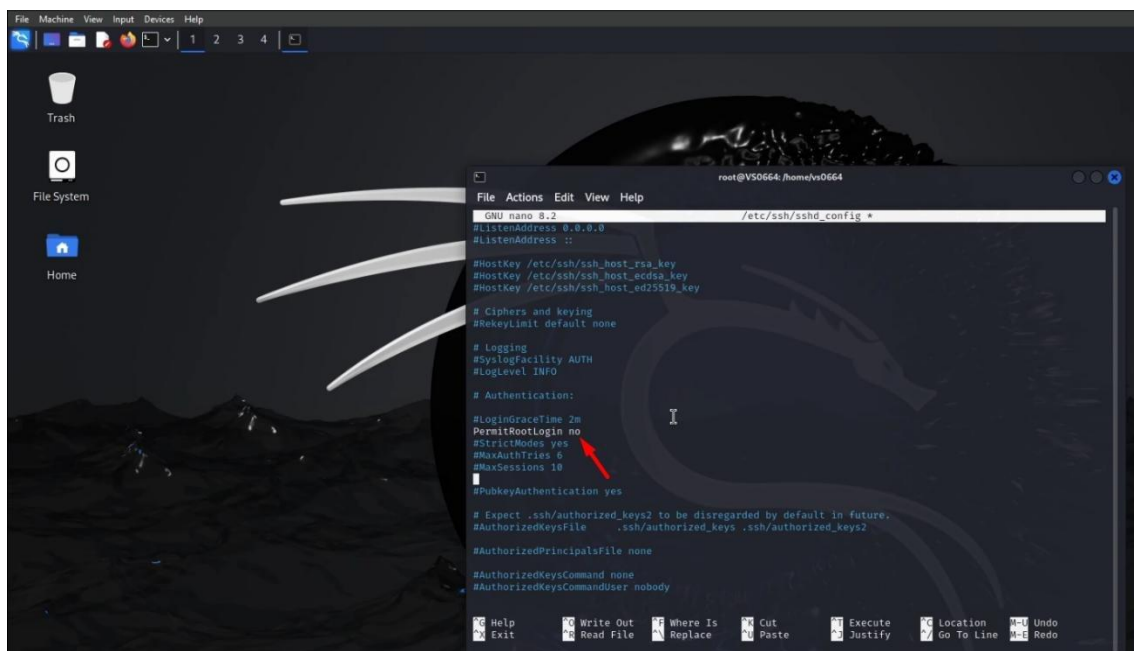
Cybersecurity / Cloud / Networks / GRC



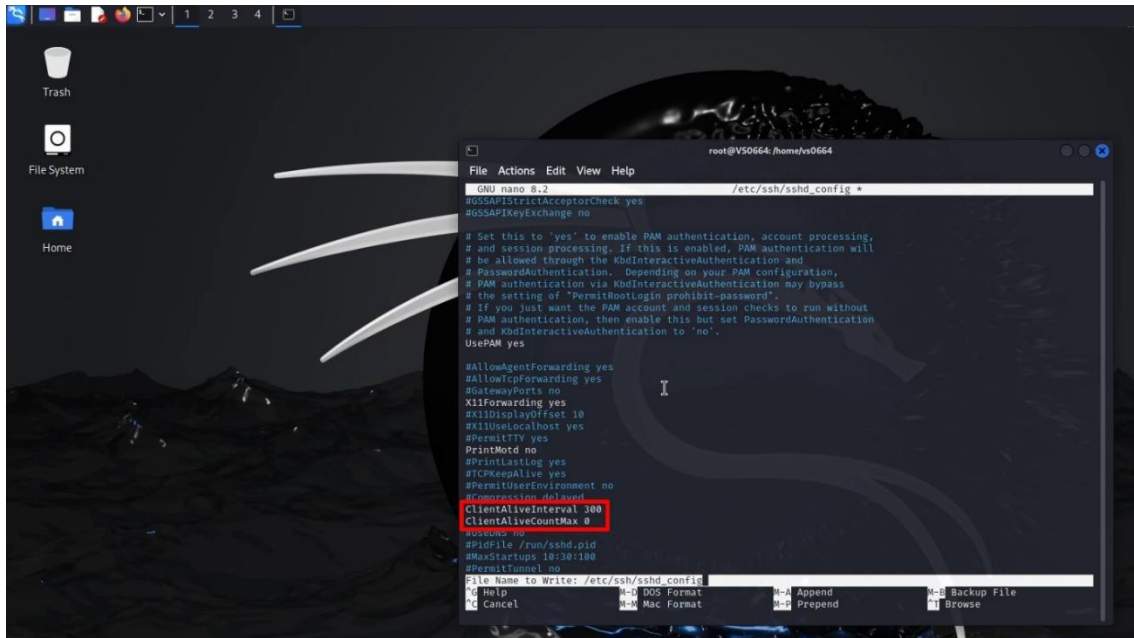
Paso 1: Accedo a la terminal de mi sistema Kali Linux como usuario root para iniciar la configuración de seguridad del servicio de acceso remoto. Ejecuto el comando `sudo nano /etc/ssh/sshd_config` para abrir el archivo de configuración principal del demonio SSH, lo que me permitirá modificar parámetros críticos de autenticación y conectividad del servidor.



Paso 2: Dentro del editor de texto nano, busco la directiva de autenticación para limitar los privilegios de entrada al sistema de forma remota. Localizo la línea `PermitRootLogin` y cambio su valor a `no`, asegurando que nadie pueda iniciar sesión directamente como superusuario a través de SSH, lo cual es una medida fundamental para mitigar ataques de fuerza bruta.



Paso 3: Me desplazo hacia el final del archivo de configuración para gestionar la persistencia de las sesiones activas y evitar conexiones fantasma. Configuro los parámetros `ClientAliveInterval 300` y `ClientAliveCountMax 0`, lo que garantiza que la sesión se cierre automáticamente tras cinco minutos de inactividad, mejorando así la gestión de recursos y la seguridad del terminal.



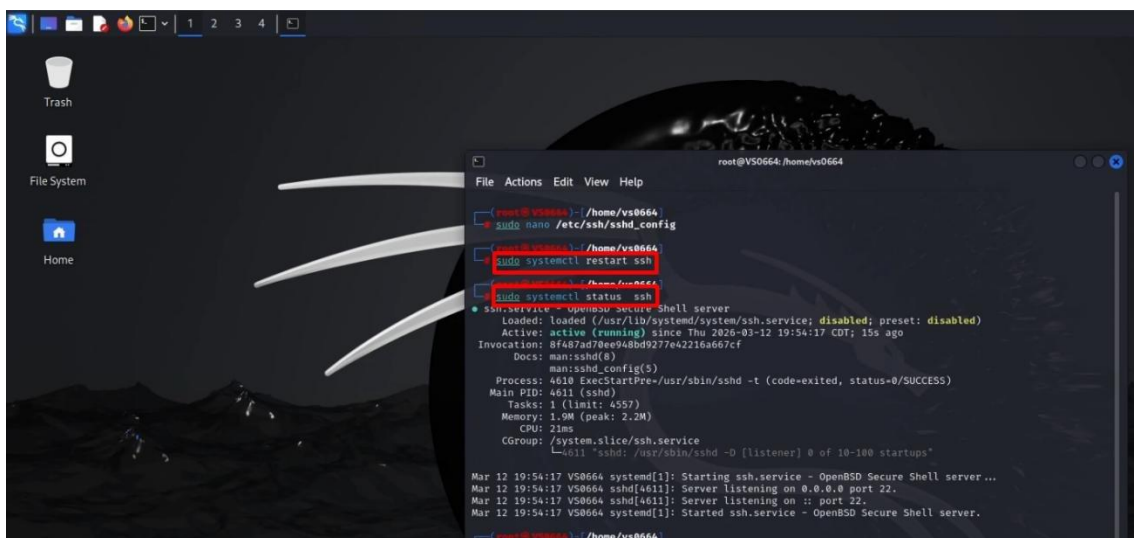
```
root@VS0664: /home/vs0664
File Actions Edit View Help
GNU nano 8.2 /etc/ssh/sshd_config
#SSAPIStrictAccepterCheck yes
#SSAPIStrictExchange no

# Set this to 'yes' to enable PAM authentication, account processing,
# and session processing. If this is enabled, PAM authentication will
# be allowed through the KbdInteractiveAuthentication and
# PasswordAuthentication. Depending on your PAM configuration,
# PAM authentication via KbdInteractiveAuthentication may bypass
# the setting of "PermitRootLogin prohibit-password".
# If you just want the PAM account and session checks to run without
# PAM authentication, then enable this but set PasswordAuthentication
# and KbdInteractiveAuthentication to 'no'.
UsePAM yes

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
#X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
#PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
ClientAliveInterval 300
ClientAliveCountMax 0
#UseDNS no
#PidFile /run/ssh.pid
#MaxStartups 10:30:100
#PermitTunnel no

All done to Write: /etc/ssh/sshd_config
Help DOS Format Append Backup File
Cancel Mac Format M-P Prepend Browse
```

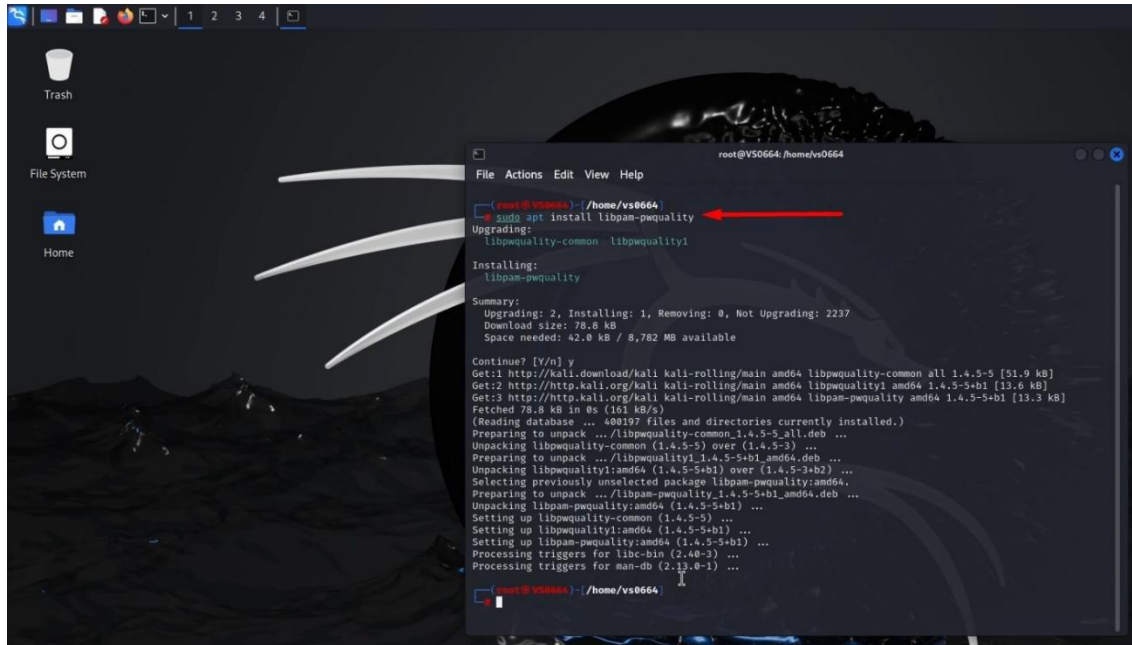
Paso 4: Tras guardar los cambios realizados en el archivo de configuración, procedo a aplicar la nueva política reiniciando el demonio mediante el comando `sudo systemctl restart ssh`. Inmediatamente después, verifico que el servicio esté corriendo correctamente sin errores utilizando `sudo systemctl status ssh`, confirmando que el estado aparece como "active (running)".



```
root@VS0664: /home/vs0664
File Actions Edit View Help
root@VS0664: /home/vs0664
# sudo nano /etc/ssh/sshd_config
root@VS0664: /home/vs0664
# sudo systemctl restart ssh
root@VS0664: /home/vs0664
# sudo systemctl status ssh
ssh.service - OpenSSH Secure Shell server
Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
Active: active (running) since Thu 2020-03-12 19:54:17 CDT; 15s ago
Invocation: 8f487ad70ee948bd9277e42216a667cf
Docs: man:sshd(8)
      man:sshd_config(5)
Process: 4610 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
Main PID: 4611 (sshd)
Tasks: 1 (limit: 4557)
Memory: 1.9M (peak: 2.2M)
CPU: 23ms
CGroup: /system.slice/ssh.service
└─4611 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

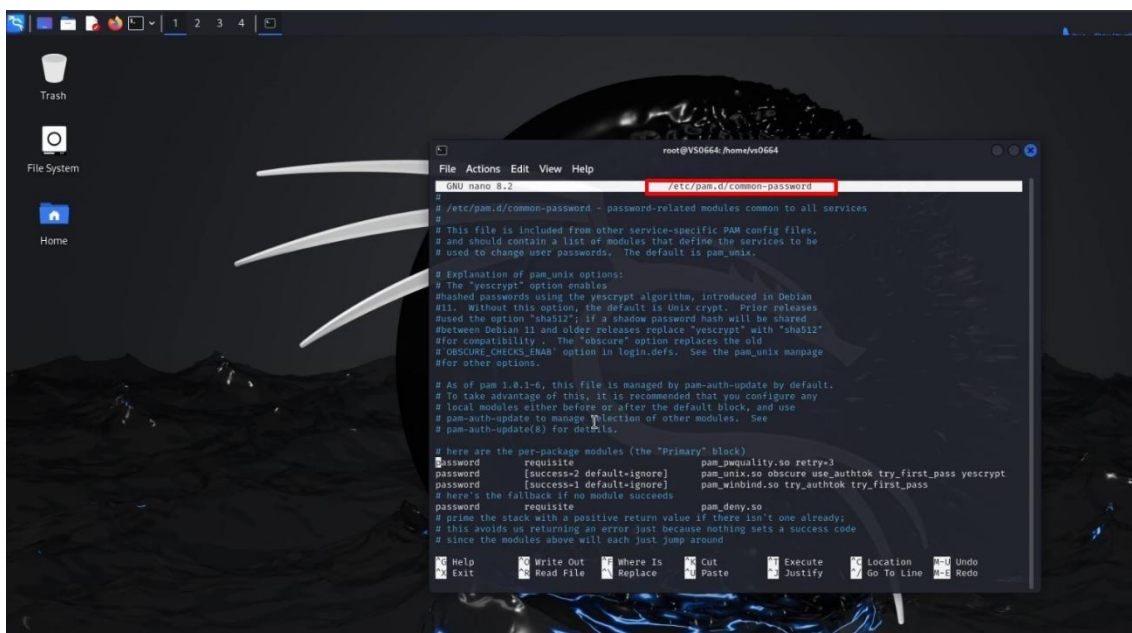
Mar 12 19:54:17 VS0664 systemd[1]: Starting ssh.service - OpenSSH Secure Shell server ...
Mar 12 19:54:17 VS0664 sshd[4611]: Server listening on 0.0.0.0 port 22.
Mar 12 19:54:17 VS0664 sshd[4611]: Server listening on :: port 22.
Mar 12 19:54:17 VS0664 systemd[1]: Started ssh.service - OpenSSH Secure Shell server.
root@VS0664: /home/vs0664
```

Paso 5: Para fortalecer la robustez de las credenciales locales, instalo una biblioteca especializada en la gestión de calidad de claves mediante el comando `sudo apt install libpam-pwquality`. Observo en la terminal cómo el gestor de paquetes descarga e instala las dependencias necesarias, preparándome para configurar reglas estrictas sobre la complejidad de las contraseñas de los usuarios.



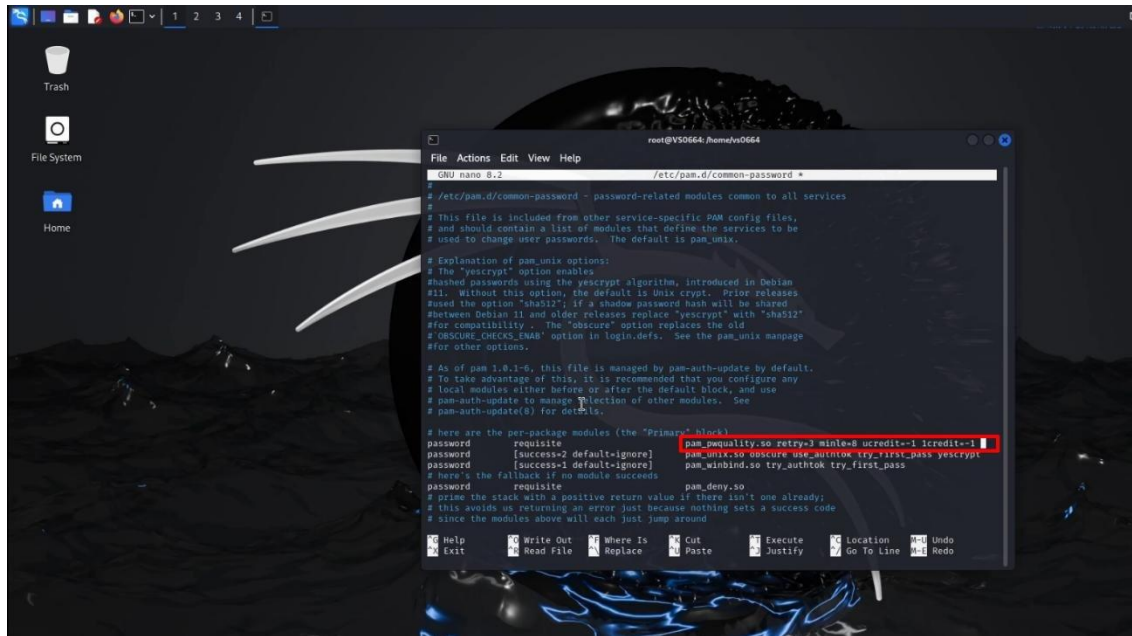
```
root@VS0664: /home/vs0664
File Actions Edit View Help
root@VS0664: /home/vs0664
$ sudo apt install libpam-pwquality
Installing:
libpam-pwquality
Summary:
Upgrading: 2, Installing: 1, Removing: 0, Not Upgrading: 2237
Download size: 78.8 kB
Space needed: 42.0 kB / 8,782 MB available
Continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 libpwquality-common all 1.4.5-5 [51.9 kB]
Get:2 http://kali.org/kali kali-rolling/main amd64 libpwquality1 amd64 1.4.5-5+b1 [13.6 kB]
Get:3 http://kali.org/kali kali-rolling/main amd64 libpam-pwquality amd64 1.4.5-5+b1 [13.3 kB]
Fetched 78.8 kB in 0s (161 kB/s)
(Reading database ... 400197 files and directories currently installed.)
Preparing to unpack .../libpwquality-common_1.4.5-5_all.deb ...
Unpacking libpwquality-common (1.4.5-5) over (1.4.5-3) ...
Preparing to unpack .../libpwquality1_1.4.5-5+b1_amd64.deb ...
Unpacking libpwquality1:amd64 (1.4.5-5+b1) over (1.4.5-3+b2) ...
Selecting previously unselected package libpam-pwquality:amd64.
Preparing to unpack .../libpam-pwquality_1.4.5-5+b1_amd64.deb ...
Unpacking libpam-pwquality:amd64 (1.4.5-5+b1) ...
Setting up libpwquality-common (1.4.5-5) ...
Setting up libpwquality1:amd64 (1.4.5-5+b1) ...
Setting up libpam-pwquality:amd64 (1.4.5-5+b1) ...
Processing triggers for libc-bin (2.13-0-1) ...
Processing triggers for man-db (2.13.0-1) ...
```

Paso 6: Abro el archivo de configuración de autenticación común del sistema ejecutando el comando `sudo nano /etc/pam.d/common-password`. En este archivo visualizo los módulos de autenticación insertables (PAM) que gestionan cómo se procesan las contraseñas en Debian, identificando la línea del módulo `pam_pwquality.so` que debo editar para aplicar mis políticas personalizadas.



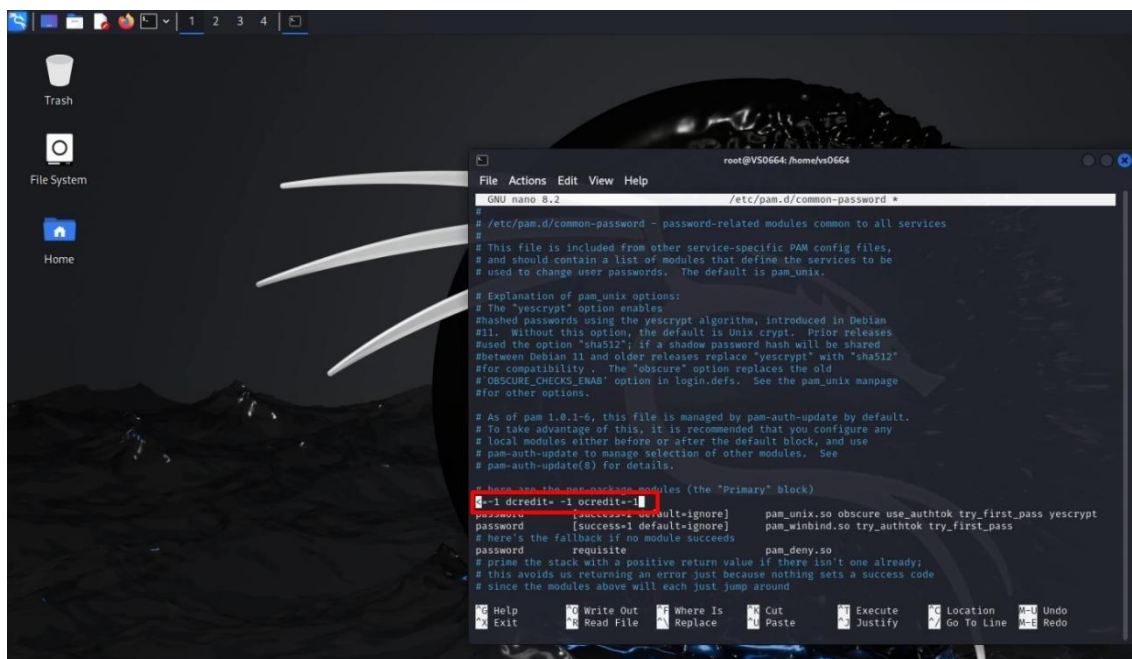
```
GNU nano 8.2 /etc/pam.d/common-password
# /etc/pam.d/common-password - password-related modules common to all services
# This file is included from other service-specific PAM config files,
# and should contain a list of modules that define the services to be
# used to change user passwords. The default is pam_unix.
# Explanation of pam_unix options:
# The "yescrypt" option enables
# hashed passwords using the yescrypt algorithm, introduced in Debian
# 11. Without this option, the default is unix crypt. Prior releases
# used the option "sha512", if a shadow password hash will be shared
# between Debian 11 and older releases replace "yescrypt" with "sha512"
# for compatibility. The "obscure" option replaces the old
# OBCRYPT_CHECK_HASH option in login.defs. See the pam_unix manpage
# for other options.
# As of pam 1.8.3-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.
# Here are the per-package modules (the "Primary" block)
password      requisite      pam_pwquality.so retry=3
password      [success=2 default=ignore] pam_unix.so obscure use_authtok try_first_pass yescrypt
password      [success=1 default=ignore] pam_unix.so try_authtok try_first_pass
# Here's the fallback if no module succeeds
password      requisite      pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
```

Paso 7: Modifico la línea del módulo pwquality para establecer requisitos técnicos rigurosos en la creación de nuevas contraseñas de usuario. Añado los parámetros `retry=3 minlen=8 ucredit=-1 lcredit=-1`, obligando a que cada clave tenga al menos ocho caracteres e incluya de forma andatoria una letra mayúscula y una letra minúscula para ser aceptada.



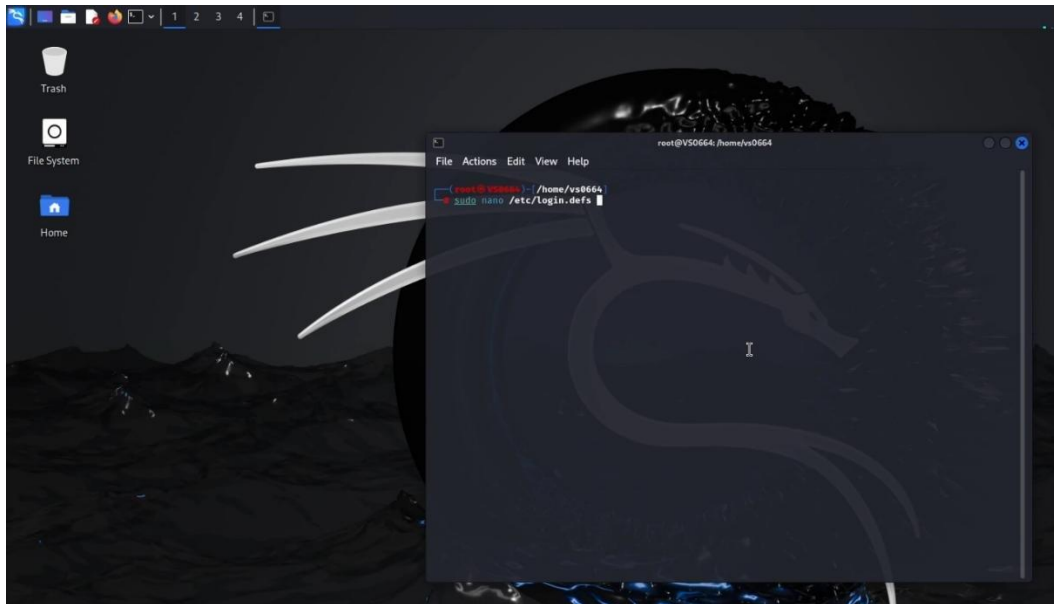
```
GNU nano 8.2 /etc/pam.d/common-password
#
# /etc/pam.d/common-password - password-related modules common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of modules that define the services to be
# used to change user passwords. The default is pam_unix.
#
# Explanation of pam_unix options:
# The "yescrypt" option enables
# hashed passwords using the yescrypt algorithm, introduced in Debian
# 11. Without this option, the default is Unix crypt. Prior releases
# used the option "sha512"; if a shadow password hash will be shared
# between Debian 11 and older releases replace "yescrypt" with "sha512"
# for compatibility. The "obscure" option replaces the old
# "OBSOLETE_CHECKS_ENAB" option in login.defs. See the pam_unix manpage
# for other options.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.
#
# here are the per-package modules (the "Primary" block)
password requisite pam_pwquality.so retry=3 minlen=8 ucredit=-1 lcredit=-1
password [success=2 default=ignore] pam_unix.so obscure use_authtok try_first_pass yescrypt
password [success=1 default=ignore] pam_winbind.so try_authtok try_first_pass
# here's the fallback if no module succeeds
password requisite pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
```

Paso 8: Continúo extendiendo la configuración de seguridad en el mismo archivo para incluir diversidad de caracteres adicionales en las credenciales. Agrego los argumentos `dcredit=-1 ocredit=-1` al final de la línea del módulo, lo que fuerza al usuario a incluir al menos un dígito numérico y un símbolo especial (como un signo de puntuación), aumentando la entropía de la clave.

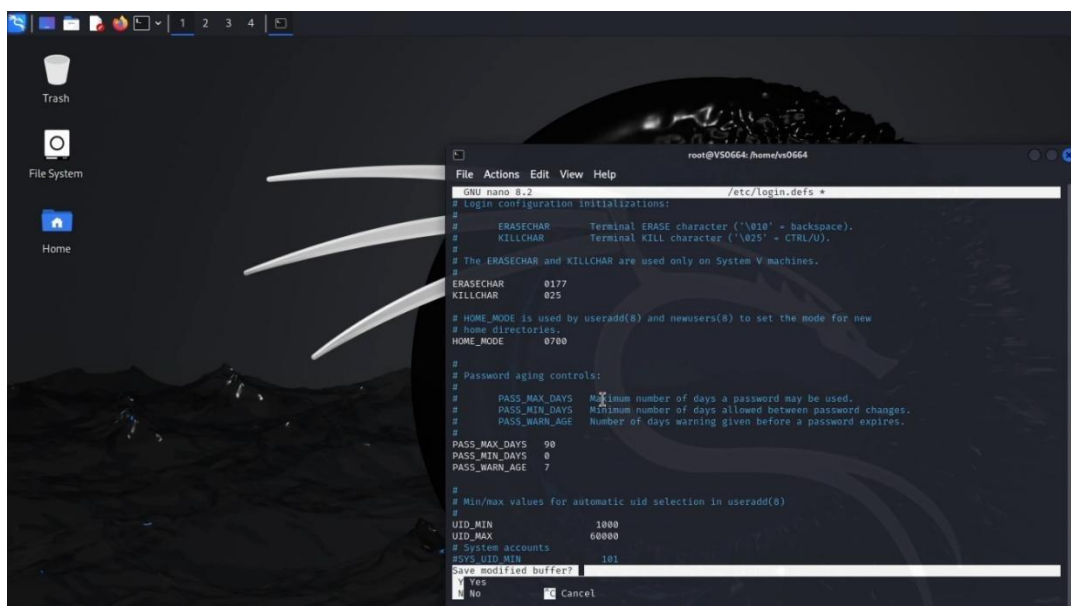


```
GNU nano 8.2 /etc/pam.d/common-password
#
# /etc/pam.d/common-password - password-related modules common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of modules that define the services to be
# used to change user passwords. The default is pam_unix.
#
# Explanation of pam_unix options:
# The "yescrypt" option enables
# hashed passwords using the yescrypt algorithm, introduced in Debian
# 11. Without this option, the default is Unix crypt. Prior releases
# used the option "sha512"; if a shadow password hash will be shared
# between Debian 11 and older releases replace "yescrypt" with "sha512"
# for compatibility. The "obscure" option replaces the old
# "OBSOLETE_CHECKS_ENAB" option in login.defs. See the pam_unix manpage
# for other options.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.
#
# here are the per-package modules (the "Primary" block)
password requisite pam_pwquality.so dcredit=-1 ocredit=-1
password [success=2 default=ignore] pam_unix.so obscure use_authtok try_first_pass yescrypt
password [success=1 default=ignore] pam_winbind.so try_authtok try_first_pass
# here's the fallback if no module succeeds
password requisite pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
```

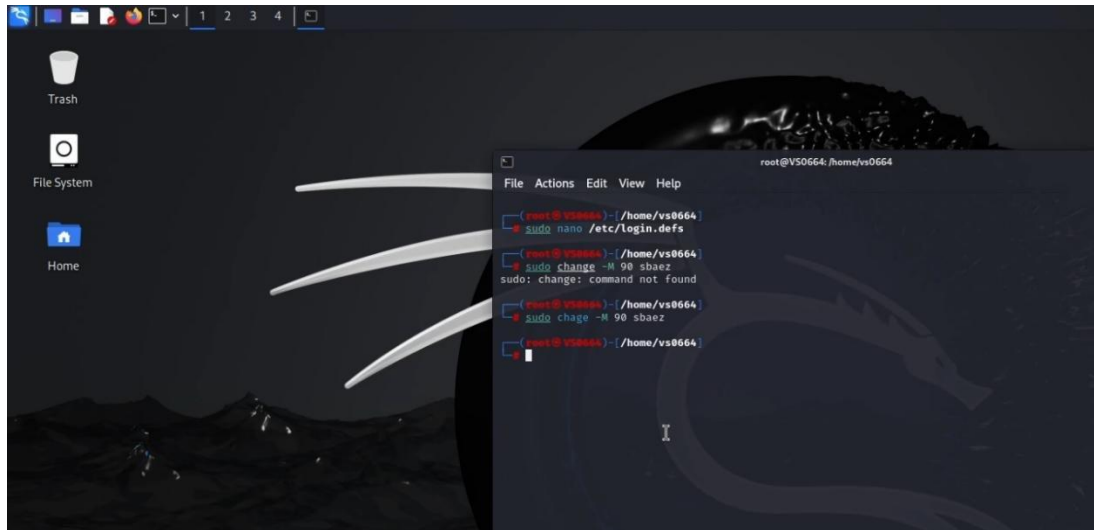
Paso 9: Utilizo nuevamente el editor para acceder a las definiciones de control de login mediante el comando `sudo nano /etc/login.defs`. Este archivo me permitirá establecer políticas globales que no solo afecten a la complejidad de la clave, sino también a su tiempo de vida y a los rangos de identificación de los usuarios creados dentro del sistema operativo.



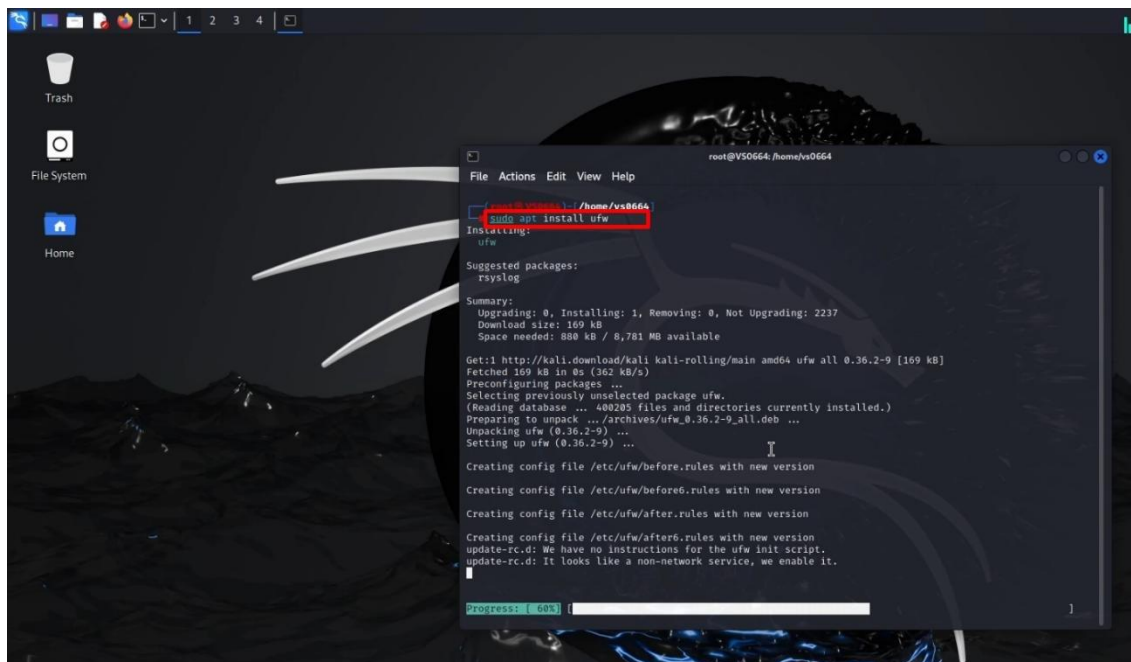
Paso 10: Dentro de `login.defs`, ajusto los controles de envejecimiento de contraseñas modificando `PASS_MAX_DAYS 90`, `PASS_MIN_DAYS 0` y `PASS_WARN_AGE 7`. Con esto, obligo a los usuarios a renovar sus claves cada noventa días y a recibir un aviso una semana antes del vencimiento, asegurando una rotación periódica de las credenciales de acceso.



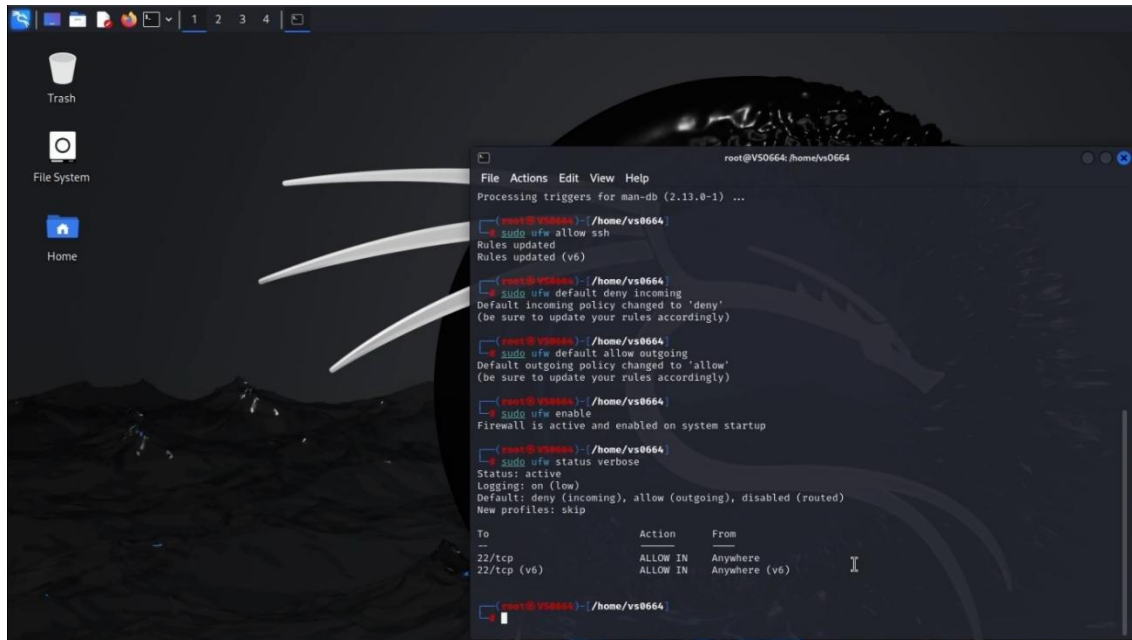
Paso 11: Aplico de forma inmediata las políticas de tiempo de vida de contraseñas a un usuario específico utilizando la herramienta de administración de cuentas. Tras un intento fallido con el comando incorrecto, ejecuto exitosamente `sudo chage -M 90 sbaez`, lo que establece que el tiempo máximo de validez de la contraseña para el usuario "sbaez" sea de noventa días exactamente.



Paso 12: Procedo a fortalecer la seguridad perimetral del sistema operativo instalando Uncomplicated Firewall mediante el comando `sudo apt install ufw`. En la terminal sigo el proceso de descarga de los paquetes necesarios y la creación de los archivos de reglas predeterminadas en el directorio `/etc/ufw`, preparando el entorno para filtrar el tráfico de red entrante y saliente.



Paso 13: Configuro las políticas de tráfico ejecutando una serie de comandos críticos: primero permito el acceso remoto con `sudo ufw allow ssh`, luego establezco una política restrictiva mediante `sudo ufw default deny incoming` y `sudo ufw default allow outgoing`. Finalmente, activo el servicio con `sudo ufw enable` y verifico su estado detallado usando `sudo ufw status verbose`.



The screenshot shows a Linux desktop environment with a dark theme. A terminal window is open, displaying the following commands and their outputs:

```
root@VS0664: /home/vs0664
Processing triggers for man-db (2.13.0-1) ...

root@VS0664: /home/vs0664# sudo ufw allow ssh
Rules updated
Rules updated (v6)

root@VS0664: /home/vs0664# sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)

root@VS0664: /home/vs0664# sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)

root@VS0664: /home/vs0664# sudo ufw enable
Firewall is active and enabled on system startup

root@VS0664: /home/vs0664# sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
```